OWN DEMAND SERVICE APP: A WEB-BASED PLATFORM FOR ON-DEMAND HOUSEHOLD SERVICES

MINI PROJECT REPORT

Submitted in partial fulfilment of the requirement for the award of the degree of

Bachelor of Technology

in

Computer Science & Engineering

Submitted to

DR. A.P.J. ABDUL KALAM TECHNICAL UNIVERSITY, UTTAR PRADESH, LUCKNOW

Submitted by

Student Name: Neeraj Manjhi
Roll No: 2301531540026
Semester & Section: 7th & E

Under the Supervision of

Dr. Alankrita Agarwal
Professor

Department of Computer Science & Engineering & Allied Branches

Lloyd Institute of Engineering & Technology

Plot No. 3, Knowledge Park II, Greater Noida, Uttar Pradesh 201306

Session: 2026–27

CERTIFICATE

This is to certify that the Mini Project Report entitled "Own Demand Service App: A Web-Based Platform for On-Demand Household Services" is a bonafide work carried out by Neeraj Manjhi, 2301531540026 in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in CSE(DS) during the academic year 2026–27, 7th semester 4th year, at the Department of Computer Science and Engineering, Lloyd Institute of Engineering and Technology.

The work embodied in this report has not been submitted elsewhere for the award of any other degree or diploma. To the best of our knowledge, the report fulfils the prescribed structure and formatting requirements of the Mini Project guidelines.

Project Guide	**Project Coordinator**	**Head of Department**
Dr. Alankrita Agarwal	Mr. Vishal Gupta	Dr. Alankrita Agarwal
Professor	Assistant Professor	Computer Science and Engineering

Date: 24/10/2026

Place: Greater Noida, U.P.

DECLARATION

I hereby declare that the Mini Project Report entitled “Own Demand Service App: A Web-Based Platform for On-Demand Household Services” submitted to the Department of Computer Science and Engineering, Lloyd Institute of Engineering and Technology, in partial fulfilment of the requirements for the degree of Bachelor of Technology in CSE(DS), is a record of original work carried out by me under the guidance of Dr. Alankrita Agarwal, Professor.

I further declare that this report has not been copied from any other source, and that all references, data and material taken from other sources have been duly acknowledged. The similarity index of this report, as checked using Chat GPT, is 10% (excluding references, code and standard boilerplate text), which is below the permitted limit of 20%.

Signature: _____

Name: Neeraj Manjhi

Roll No.: 2301531540026

Date: 24/10/2026

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my project guide, Dr. Alankrita Agarwal , Professor, Department of Computer Science and Engineering, for the valuable guidance, constant encouragement and constructive suggestions throughout the development of this mini project.

I am thankful to Mr. Vishal Gupta, Project Coordinator, and Dr. Alankrita Agarwal , Head of the Department of Computer Science and Engineering, for providing the necessary facilities and support. I also extend my thanks to Dr. Kumud Saxena, Lloyd Institute of Engineering and Technology, for providing a conducive academic environment.

Finally, I thank all the faculty members, laboratory staff, my family and friends for their continuous support and motivation during the course of this work.

ABSTRACT

People who move to a new locality may not know reliable cleaners, electricians, plumbers, appliance repair technicians, mechanics or other local service providers. Finding help through informal contacts can take time and may provide little visibility into the service request or its status. The Own Demand Service App is a web-based concept intended to organise common household-service requests through a single interface.

A customer selects a service category, enters basic contact and location details, chooses a preferred date and time, describes the work and submits a booking request. The interface then creates a demo booking identifier and presents a summary with a sample status progression. The prototype provides categories for home cleaning, plumbing, electrician work, appliance repair, salon/personal care and other services. It validates required fields and a ten-digit phone number, prevents selection of a past date in the browser, and escapes user-entered text before displaying it in the confirmation area.

The current implementation is a client-side academic demonstration using HTML, CSS and JavaScript. It does not use a server, persistent database, provider directory, authentication, payment gateway or real dispatch mechanism; therefore, a displayed confirmation is not a real-world service booking.

The project demonstrates the interface flow and provides a foundation for future development involving secure accounts, verified providers, location-aware discovery, booking persistence, ratings, notifications and administrative controls. The main contribution is a simple, consolidated workflow for expressing service needs, especially for people unfamiliar with their locality.

Keywords: on-demand services, household services, web application, booking interface, service management, front-end prototype.

TABLE OF CONTENTS

CHAPTER 1: INTRODUCTION **1**

1.1 Background 1

1.2 Problem Statement 2

1.3 Objectives 3

1.4 Scope of the Project 4

1.5 Organisation of the Report 5

CHAPTER 2: LITERATURE REVIEW / EXISTING SYSTEM **6**

2.1 Review of Related Work 6

2.2 Existing System 7

2.3 Gaps Identified 8

2.4 Proposed System 9

2.5 User Personas and Usage Scenarios 10

2.6 Technical Source Grounding 11

CHAPTER 3: SYSTEM ANALYSIS AND DESIGN **12**

3.1 Requirement Analysis 12

3.1.1 Functional Requirements 12

3.1.2 Non-Functional Requirements 12

3.1.3 Hardware and Software Requirements 13

3.2 Feasibility Study 14

3.3 System Architecture 15

3.4 Design Diagrams 16

3.4.1 Data Flow Diagram 16

3.4.2 ER / UML Diagrams 16

3.5 Technology Stack 18

3.6 Use-Case Descriptions 19

3.7 Data Dictionary 20

3.8 Interface Design Principles 21

CHAPTER 4: IMPLEMENTATION **22**

4.1 Module Description 22

4.1.1 Landing Page and Service Cards 22

4.1.2 Booking Form 22

4.1.3 Date and Time Selection 22

4.1.4 Booking Creation and Confirmation 23

4.1.5 Status Tracking and Reset 23

4.2 Key Algorithms 24

4.3 Code Snippets 25

4.4 Screenshots 27

4.5 Error Handling and Input Safety 28

4.6 Deployment and Maintenance 29

CHAPTER 5: TESTING 30

5.1 Testing Strategy 30

5.2 Test Environment and Preconditions 31

5.3 Test Cases and Test Results 32

5.4 Sample Input / Output 33

5.5 Boundary and Negative Testing 34

5.6 Test Evidence and Acceptance Criteria 35

5.7 Test Execution Record Template 36

CHAPTER 6: RESULTS AND DISCUSSION 37

6.1 Output Screenshots 37

6.2 Performance / Accuracy Analysis 39

6.3 Discussion 40

6.4 Observed and Unimplemented Capabilities 41

6.5 Risk and Mitigation 42

6.6 Evaluation Metrics for a Future Study 43

6.7 Privacy, Trust and Service Quality 44

CHAPTER 7: CONCLUSION AND FUTURE SCOPE 45

7.1 Conclusion 45

7.2 Limitations 46

7.3 Future Scope 47

7.4 Final Remarks 48

REFERENCES 49

APPENDIX 50

Appendix A: Source Code 50

Appendix B: Additional Material 51

LIST OF FIGURES

Figure	Caption
Fig. 3.1	High-level system architecture
Fig. 3.2	Booking request data flow (level 0)
Fig. 3.3	Use-case view of the prototype
Fig. 3.4	Conceptual future entity relationship model
Fig. 4.1	Service request processing workflow
Fig. 4.2	Demonstration status progression
Fig. 4.3	Landing page with service cards
Fig. 4.4	Booking form filled with dummy data
Fig. 6.1	Demo confirmation with generated booking ID
Fig. 6.2	Status panel after a sample status change
Fig. 6.3	Validation warning for an invalid phone number

LIST OF TABLES

Table	Caption
Table 1.1	Organisation of the report
Table 2.1	Comparison of approaches to finding household services
Table 2.2	Gaps identified and prototype response
Table 2.3	Intended user personas
Table 3.1	Functional requirements
Table 3.2	Non-functional requirements
Table 3.3	Hardware and software requirements
Table 3.4	Use-case descriptions
Table 3.5	Booking object fields
Table 4.1	Prototype modules
Table 5.1	Test cases and expected outcomes
Table 5.2	Phone-field boundary cases
Table 5.3	Test execution record
Table 6.1	Observed and unimplemented capabilities
Table 6.2	Risk and mitigation summary
Table 7.1	Development roadmap
Table A.1	Source elements

LIST OF ABBREVIATIONS

Abbreviation	Full form
API	Application Programming Interface
CSS	Cascading Style Sheets
DFD	Data Flow Diagram
DOM	Document Object Model
ER	Entity Relationship
FR	Functional Requirement
HTML	HyperText Markup Language
ID	Identifier
NFR	Non-Functional Requirement
ODS	Own Demand Service (project prefix)
OWASP	Open Worldwide Application Security Project
RAM	Random Access Memory
TC	Test Case
UI	User Interface
UML	Unified Modeling Language
W3C	World Wide Web Consortium
WCAG	Web Content Accessibility Guidelines
WHATWG	Web Hypertext Application Technology Working Group

CHAPTER 1: INTRODUCTION

1.1 Background

Household services are recurring practical needs. A resident may require cleaning, plumbing, electrical maintenance, appliance repair, vehicle repair or personal-care assistance. A person who has recently moved to a locality may not have neighbours, family members or established contacts who can recommend a suitable provider. Searching across multiple sources can be inconvenient, and requests may be lost in informal calls or messages.

The Own Demand Service App explores a single web interface where a user can state what help is required and provide a preferred schedule and locality. The interface is designed around a familiar sequence: choose a category, enter details, submit the request and view a status summary.

The academic prototype focuses on this user journey rather than claiming to solve provider supply, trust, payment or logistics at production scale. Its main features are:

- Centralised service categories in one interface.
- Structured request details with validation.
- A visible demo booking identifier and a sample status summary.

1.2 Problem Statement

The problem addressed is the lack of a simple, consolidated starting point for requesting different everyday services, particularly for users unfamiliar with their new neighbourhood. Existing informal approaches may require separate searches and conversations for each service. They may also leave users uncertain about what information to provide or whether a request has been recorded.

The project question is: how can a lightweight website present multiple household-service categories and collect a consistent service request in a clear, responsive workflow? The prototype demonstrates a possible interface-level answer. It does not independently guarantee service availability, provider quality or booking fulfilment.

1.3 Objectives

The primary objective is to develop a responsive front-end prototype that demonstrates a structured household-service request workflow. The project also aims to make the interface understandable to a first-time user and to show how a request could be tracked after submission.

- Present multiple service categories in one interface.
- Collect customer name, phone, date, time, locality and a description of the work.
- Validate mandatory information and the ten-digit phone format.
- Generate a demo reference number for each request.
- Display a booking summary and simulated status changes.
- Use responsive styling for common desktop and mobile screen widths.
- Document limitations and a practical path to a secure, full-stack system.

1.4 Scope of the Project

The implemented scope is limited to a browser-run demonstration. The project uses a single HTML page with embedded CSS and JavaScript. The user can select one of the provided service categories, complete the form, create a demo booking and change the displayed sample status using demonstration buttons.

The following items are outside the current implementation and are described only as future enhancements:

- Included: six service categories, booking form, client-side validation, demo booking ID, sample status steps and form reset.
- Excluded: real provider search and availability, user login, server-side validation, database storage, map/geolocation services, real notifications, online payments, provider identity verification, ratings and an administrative portal.

1.5 Organisation of the Report

The report is organised into seven chapters followed by references and appendices, as summarised in Table 1.1.

Table 1.1: Organisation of the report

Chapter	Content
Chapter 1	Introduces the problem, objectives and scope.
Chapter 2	Reviews common ways of finding local services and identifies the gaps addressed by the prototype.
Chapter 3	Documents requirements, feasibility, architecture, diagrams and interface design.
Chapter 4	Explains the implemented modules, algorithms, code snippets and input safety.
Chapter 5	Describes the testing strategy, test cases and the evidence to be recorded.
Chapter 6	Discusses the prototype results, risks and limitations.
Chapter 7	Concludes the work and proposes future enhancements.
References / Appendices	List the sources consulted, a source-code overview and the user guide.

CHAPTER 2: LITERATURE REVIEW / EXISTING SYSTEM

2.1 Review of Related Work

Digital service marketplaces commonly organise service categories, customer requests and provider responses. A typical marketplace may include provider profiles, service descriptions, pricing, availability, booking management, reviews and customer support. These capabilities establish a useful conceptual reference for the problem, but the submitted demo is intentionally much smaller and should not be confused with a commercial marketplace.

Web application design guidance generally encourages clear labels, understandable validation messages, responsive layouts and predictable feedback after user actions. These principles are relevant to the prototype because the intended user may be new to the area and may not know which details a service provider will need. A structured form can reduce ambiguity by collecting a category, time preference and brief work description.

2.2 Existing System

One approach is word-of-mouth discovery through neighbours, friends or building staff. It can offer personal recommendations but is less available to someone who has just moved. A second approach is searching the web or local listings, which may provide multiple options but requires the user to compare providers and contact them separately. A third approach is using separate specialist applications for particular service categories. This can work well where available, but users may need to switch between platforms for unrelated tasks.

The project does not assert that these approaches are universally inadequate. Instead, it focuses on the specific usability opportunity of presenting several everyday service categories and one consistent request form in a simple demonstration. Table 2.1 compares the approaches.

Table 2.1: Comparison of approaches to finding household services

Approach	Strengths	Limitations
Word of mouth	Personal recommendations; trusted contacts	Not available to a new resident; no record of the request
Web search / local listings	Many options; easy to start	Providers must be compared and contacted separately; details repeated
Specialist apps	Strong for one category; provider support	Different apps for unrelated services; inconsistent forms
Full marketplace	Provider search, pricing, reviews, tracking	Requires backend, verification and operations; beyond a mini project
Own Demand prototype	One interface; fixed fields; demo ID and status	Front-end only: no providers, database, accounts or real dispatch

2.3 Gaps Identified

The intended gaps are primarily workflow-related: users may have to repeat basic details across services; requests may be recorded in different formats; and a new resident may lack a clear starting point. A single interface can make the request process more consistent, while a reference number and visible status can demonstrate the concept of request tracking.

The current prototype only simulates tracking in the browser. Because no database or backend exists, the booking is not durable across page reloads and no provider receives it. Consequently, the gap analysis is a design rationale, not a measured comparative study of real services.

Table 2.2: Gaps identified and prototype response

Gap	Prototype response
No unified front-end entry point in the proposed user journey	Six categories presented on one page
Inconsistent details when requests are made informally	One structured form with required fields and validation
Limited visibility of request state in an unstructured conversation	Demo booking ID and four-step status display
Trust and fulfilment need verification and operational processes	Not addressed by UI design; listed as future scope

2.4 Proposed System

The proposed system is the Own Demand Service App. A customer opens the website, selects a service category, enters required information and submits the form. Client-side JavaScript checks the category and form validity, generates a demo identifier and renders a confirmation summary. The user can then demonstrate status transitions to Confirmed, In Progress and Completed.

In a production version, a server would validate the request, store it in a database, identify relevant providers, deliver the request to those providers and update status based on authenticated actions. Those production functions are recommendations only and are not included in the submitted ZIP.

2.5 User Personas and Usage Scenarios

Three personas guide the design. They are design assumptions, not findings from formal user interviews, and should be validated with real users before the system is developed further.

Table 2.3: Intended user personas

Persona	Need	Design response
New resident	A clear starting point without a trusted local network	Simple categories; no trade terminology needed
Busy resident	Give the same basic details quickly and consistently	One reusable form for every category
Student or tenant	Occasional cleaning, plumbing or electrical help in an unfamiliar market	Optional free-text description of the task

A typical scenario begins when the user opens the page and selects Home Cleaning. They enter dummy contact information, choose a preferred date and time, enter a sample locality and describe the task. On submission, the interface creates a demo ID and shows a status. Another scenario is an electrical maintenance request: the same form is reused, which illustrates consistency across categories. The system currently does not identify an available electrician or confirm a real appointment.

2.6 Technical Source Grounding

The following sources connect the report discussion to the References section.

- [1], [2] The HTML Living Standard and HTML forms documentation inform the use of semantic form controls, required fields, input types and browser validation. These controls do not replace server-side validation.
- [3] The CSS reference supports the service-card layout, spacing and responsive presentation of the single-page interface.
- [4] The JavaScript Guide is relevant to event handlers, category selection, form submission and dynamic rendering described in Chapter 4. The project uses browser-side JavaScript only.
- [5] The OWASP guidance is relevant to treating user-entered strings as untrusted. The prototype escapes text before rendering its confirmation summary, but this limited front-end measure is not a complete security design.
- [6] WCAG 2.2 is a reference for future accessibility checks, including keyboard access, focus visibility, contrast and understandable form feedback. No formal conformance audit is claimed.
- [7] The submitted source archive contains the front-end files described in Appendix A.

These sources support the stated technology and design discussion; they do not establish measured performance, user-study results or production security for the prototype.

CHAPTER 3: SYSTEM ANALYSIS AND DESIGN

3.1 Requirement Analysis

Requirement analysis identifies the behaviour that a user should be able to observe and the quality attributes that guide the interface. The requirements below are derived from inspection of the submitted index.html and its accompanying README. They describe the front-end demo rather than a deployed service platform. Each requirement can be demonstrated manually in a browser.

3.1.1 Functional Requirements

Table 3.1: Functional requirements

ID	Functional requirement	Priority
FR-01	Display the Own Demand Service App heading and a notice that the interface is a demo.	High
FR-02	Display service cards for Home Cleaning, Plumbing, Electrician, Appliance Repair, Salon / Personal Care and Other Service.	High
FR-03	Highlight the selected service category.	High
FR-04	Require customer name, phone number, preferred date, preferred time and address/locality.	High
FR-05	Accept a brief optional description of the required work.	Medium
FR-06	Validate a ten-digit numeric phone number.	High
FR-07	Set the minimum selectable date to the browser's current local date.	Medium
FR-08	Generate a demo booking ID when the form is valid.	High
FR-09	Display the entered booking details and current status.	High
FR-10	Allow the demonstration status to be changed to Confirmed, In Progress or Completed.	Medium
FR-11	Clear the form and reset the visible booking state.	Medium

3.1.2 Non-Functional Requirements

Non-functional requirements concern usability, compatibility, safety and maintainability. The current demo offers responsive CSS layout and client-side input checks. Its security and availability properties are not equivalent to those of a server-backed application.

Table 3.2: Non-functional requirements

ID	Requirement	Category
NFR-01	Labels and steps should be clear to a first-time user.	Usability
NFR-02	Service cards and form fields should reflow at narrow widths.	Responsiveness
NFR-03	HTML structure, CSS styling and JavaScript behaviour should remain understandable.	Maintainability
NFR-04	User-entered strings are escaped before insertion into the confirmation summary.	Input safety
NFR-05	Target modern browsers supporting standard HTML5, CSS and JavaScript.	Compatibility
NFR-06	Use dummy details during demonstration; no secure data store is provided.	Privacy
NFR-07	Validation should prevent common incomplete or malformed submissions.	Reliability

3.1.3 Hardware and Software Requirements

The project is lightweight and runs locally in a modern web browser. No server installation or database configuration is required for the submitted demonstration. The values in Table 3.3 are indicative minimums.

Table 3.3: Hardware and software requirements

Category	Item	Requirement
Hardware	Processor	Dual-core, 1.5 GHz or higher
Hardware	RAM	4 GB minimum
Hardware	Storage	Under 5 MB for the project files
Hardware	Display	Desktop or mobile screen (responsive layout)
Software	Operating system	Windows, Linux, macOS, Android or iOS
Software	Browser	Current Chrome, Edge or Firefox
Software	Languages	HTML5, CSS3 and vanilla JavaScript
Software	Development tools	Text editor (e.g. VS Code) and browser developer tools
Software	Server / database	Not required
Network	Internet access	Not required to run the demo

3.2 Feasibility Study

Technical feasibility is high for the demo because it uses standard browser technologies. HTML provides page structure and form controls; CSS provides visual layout and responsive behaviour; JavaScript handles category selection, form submission, validation, booking ID creation and status rendering. The absence of a backend simplifies deployment but also prevents persistent storage and multi-user workflows.

Economic feasibility is favourable for a student prototype because the demo can be run from a local file with minimal direct operating cost. A production system would incur development, hosting, database, monitoring, notification, support and provider-verification costs, which have not been estimated here.

Operational feasibility is suitable for demonstrating the request flow without training. For real services it is not established, because provider supply, scheduling conflicts, cancellations, disputes and emergency support are not implemented.

- Technical: feasible for the intended academic scope.
- Economic: negligible cost for the demo; production costs not estimated.
- Operational: usable as a classroom demonstration; real-world operation requires onboarding, support and policies.

3.3 System Architecture

The current architecture is a single-page client-side application. The browser loads index.html, renders the interface and executes embedded JavaScript. State is held in memory through the selectedService and currentBooking variables, and a page reload clears this state. Fig. 3.1 summarises the architecture.

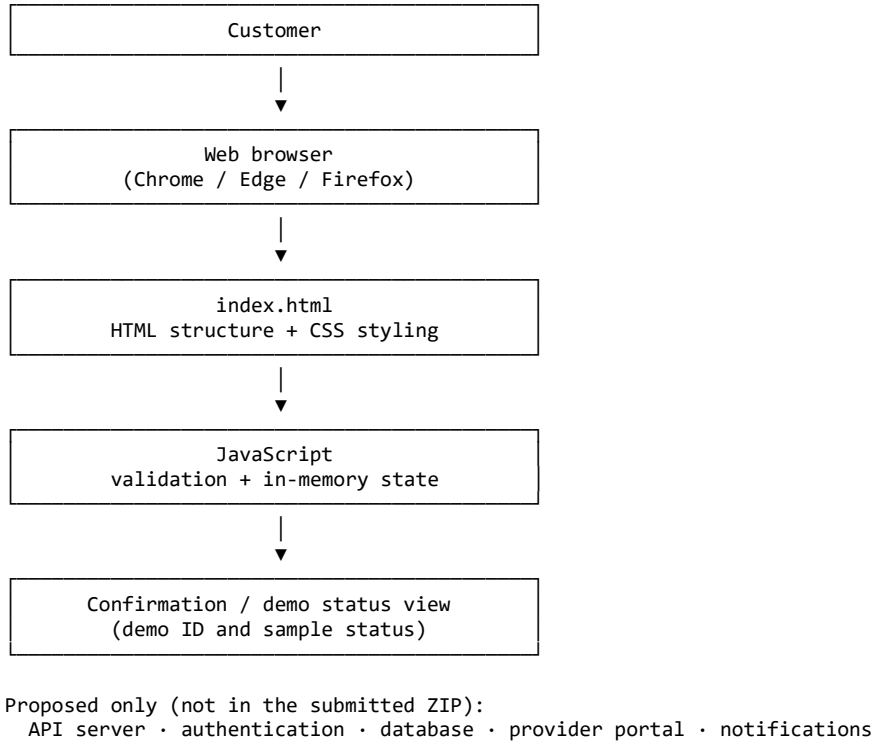


Fig. 3.1: High-level system architecture

The future production architecture could add a web client, application API, authentication service, relational database, provider portal, notification service and administrator console. These components are conceptual and are not present in the ZIP.

3.4 Design Diagrams

3.4.1 Data Flow Diagram

Fig. 3.2 shows the level-0 data flow. The customer provides a service choice and booking details; the prototype validates the input, creates an in-memory demo booking and shows the confirmation and sample status.

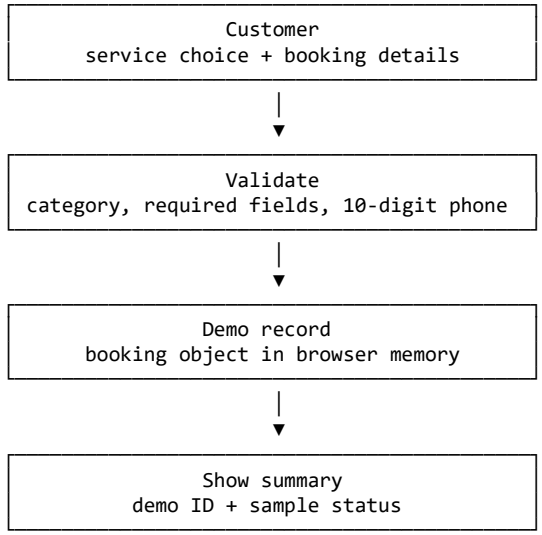


Fig. 3.2: Booking request data flow (level 0)

3.4.2 ER / UML Diagrams

Fig. 3.3 gives the use-case view of the prototype. In the current prototype, the status buttons are available for demonstration rather than as role-protected provider actions.

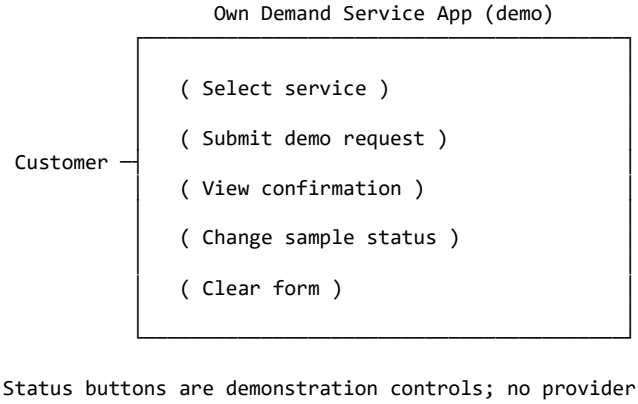


Fig. 3.3: Use-case view of the prototype

Fig. 3.4 shows a conceptual future entity relationship model. It is a proposal only; these entities are not stored in the current implementation.

```
CUSTOMER (1) ---------- places ---------- (0..*) BOOKING
SERVICE_CATEGORY (1) -- classifies ------- (0..*) BOOKING
PROVIDER (0..1) ------- fulfils ---------- (0..*) BOOKING
BOOKING (1) ----------- receives --------- (0..1) REVIEW

CUSTOMER: customer_id, name, phone
SERVICE_CATEGORY: category_id, title
PROVIDER: provider_id, name, verification_status, service_area
BOOKING: booking_id, customer_id, category_id, provider_id,
         preferred_date, preferred_time, address, description, status
REVIEW: review_id, booking_id, rating, comment
```

Fig. 3.4: Conceptual future entity relationship model

3.5 Technology Stack

HTML5 structures the page and provides semantic form controls [1], [2]. CSS styles the colour palette, card grid, spacing, confirmation panel and responsive breakpoints [3]. Vanilla JavaScript manages the selection and form workflow [4]. The prototype does not use a framework, external API, build tool, backend runtime or database.

- Structure: HTML5 single page with a booking form.
- Presentation: embedded CSS with an auto-fit card grid and a media query for narrow screens.
- Behaviour: embedded vanilla JavaScript with event handlers and DOM updates.
- Runtime: any modern web browser; no server required.

3.6 Use-Case Descriptions

Table 3.4: Use-case descriptions

Use case	Actor	Main flow	Alternate flow / postcondition
Select service	Customer	Customer clicks a service card; the card is highlighted and its category recorded.	Selecting another card moves the highlight. Postcondition: one category is held in browser memory.
Submit demo request	Customer	Customer submits the form; browser validation runs; JavaScript checks category and phone; booking object is created; confirmation and status panel are rendered.	Missing category or invalid phone shows a warning and prevents booking creation. Postcondition: request exists in the current page session only.
Change sample status	Demonstration user	User presses a status button and the displayed status is updated.	Not a provider workflow: any person using the page can change the sample status. A production system would restrict this to an authenticated role.

3.7 Data Dictionary

The current booking object exists in JavaScript memory and is not written to disk or transmitted to a server. Its fields are listed in Table 3.5.

Table 3.5: Booking object fields

Field	Type	Description
id	String	Generated identifier beginning with ODS followed by a random five-digit number.
name	String	Trimmed customer name entered in the form.
service	String	Selected service category.
date	String	Chosen preferred date.
time	String	Selected time-window label.
address	String	Locality or address entered by the user.
issue	String	Optional description of the work.
status	String	Current demo status, initially "Booking Requested".

A production data model should use stable primary keys and server-generated identifiers rather than a random client-side number. It should distinguish requested time from confirmed appointment time, record creation and update timestamps, keep provider assignment nullable until a provider accepts, and make status changes auditable with actor, timestamp and prior/new status. Address data should be collected only to the level needed and protected from unauthorised access.

3.8 Interface Design Principles

The interface uses a prominent header, a short explanatory introduction, a grid of service cards, a booking form and a status panel. This order mirrors the user's task sequence and reduces the need to navigate between pages. Service icons help distinguish categories visually, while labels provide the meaning in text, and a card highlight communicates the currently selected option. Form labels are connected to their controls through matching for/id values, and required fields use native browser validation.

The responsive grid uses auto-fit and a minimum card width, while a media query adjusts the header on narrow screens. Further accessibility work [6] should verify keyboard-only operation, focus visibility, colour contrast, screen-reader announcements, zoom behaviour and error-message clarity. A formal accessibility evaluation has not been performed.

CHAPTER 4: IMPLEMENTATION

4.1 Module Description

The implementation is contained in Own_Demand_Service_App_Demo/index.html. The README explains how to extract the ZIP and open the file in Chrome, Edge or another modern browser. The modules in Table 4.1 are logical responsibilities within the single page rather than separately deployed software components.

Table 4.1: Prototype modules

Module	Responsibility	Key elements
Landing page	Project name, "Student Demo Prototype" badge and demo notice	Header and hero section
Service cards	Six selectable categories with highlight	data-service, selectedService
Booking form	Collects customer and schedule details	bookingForm
Date and time	Minimum date and four time windows	Date input, time dropdown
Booking creation	Validation, object creation, confirmation	Submit handler, renderBooking()
Status tracking	Four-step progression and sample status buttons	data-status buttons
Reset	Clears form and in-memory state	resetBtn handler
Input safety	Escapes user text before rendering	escapeHTML()

4.1.1 Landing Page and Service Cards

The header presents the project name and a "Student Demo Prototype" badge. The hero section explains that the page collects a request but does not contact real providers. Six service cards let the customer choose a category. A click handler removes the active style from all cards, adds it to the clicked card and stores its data-service value in selectedService.

4.1.2 Booking Form

The form requests customer name, phone number, preferred date, preferred time, address/locality and an optional service description. HTML required attributes provide basic browser validation. The phone field uses a numeric input mode, a ten-character maximum and a pattern for ten digits. JavaScript repeats the phone format check before creating a booking.

4.1.3 Date and Time Selection

The preferred date uses an HTML date input. JavaScript computes the browser-local current date by adjusting for the timezone offset and assigns it as the minimum date. The time dropdown offers four windows: 9 AM-12 PM, 12 PM-3 PM, 3 PM-6 PM and 6 PM-8 PM. These are user preferences for the demo, not confirmed provider availability.

4.1.4 Booking Creation and Confirmation

When the form is submitted, the default browser submission is prevented. The script checks that a category has been selected and validates the phone number. It creates an in-memory object containing a generated ID with the ODS prefix, customer details, service category, schedule, address, issue description and the initial status "Booking Requested". The confirmation panel renders these values and repeats that the booking is demo-only.

4.1.5 Status Tracking and Reset

The status panel shows the booking ID, selected service and a four-step visual progression: request received, confirmed, in progress and completed. Buttons set the sample status and call the renderer again. The Clear Form button resets the form, clears selectedService and currentBooking, removes category highlighting, hides the confirmation and status buttons, and restores the empty status message.

4.2 Key Algorithms

```
Algorithm 1: Service selection
  FOR each service card: remove class "active"
  add class "active" to the clicked card
  selectedService <- data-service value of the clicked card

Algorithm 2: Submission
  prevent default form submission
  IF no service is selected: show warning; STOP
  IF phone is not exactly 10 digits: show warning; STOP
  create booking object (id, details, status = "Booking Requested")
  render confirmation and status steps

Algorithm 3: Status update
  IF no booking exists: show warning; STOP
  currentBooking.status <- status of the clicked button
  redraw summary

Algorithm 4: Reset
  reset the native form
  clear selectedService and currentBooking
  remove card highlight; hide confirmation and status buttons
  restore the empty status message
```

Fig. 4.1 gives the overall request-processing workflow and Fig. 4.2 the demonstration status progression.

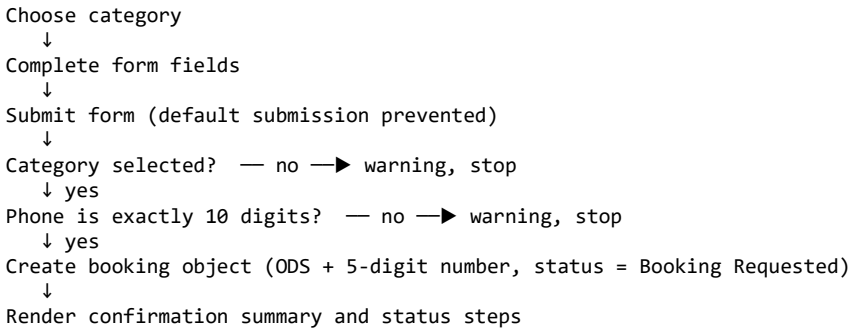


Fig. 4.1: Service request processing workflow

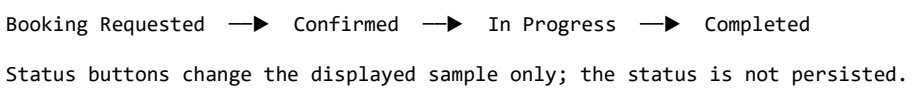


Fig. 4.2: Demonstration status progression

4.3 Code Snippets

Service card selection uses a click listener on each card. The implementation removes the active class from all cards and then applies it to the chosen card, so only one category is selected at a time.

```
let selectedService = "";
let currentBooking = null;

cards.forEach(card => {
  card.addEventListener("click", () => {
    cards.forEach(c => c.classList.remove("active"));
    card.classList.add("active");
    selectedService = card.dataset.service;
  });
});
```

Phone validation uses the regular expression below. It accepts exactly ten digits and rejects alphabetic characters, spaces and values with fewer or more than ten digits after trimming. This is client-side validation only; a production server must validate all inputs independently.

```
const phone = phoneInput.value.trim();
if (!/^[0-9]{10}$/.test(phone)) {
  alert("Please enter a valid 10-digit phone number.");
  return;
}
```

The escapeHTML helper replaces ampersand, less-than, greater-than, double quote and apostrophe characters with HTML entities before user-entered values are interpolated into the confirmation markup [5]. This reduces the risk of user text being interpreted as markup in that rendering path, but it does not replace server-side security controls.

```
function escapeHTML(value) {
  return String(value)
    .replace(/&/g, "&amp;")
    .replace(/</g, "&lt;")
    .replace(/>/g, "&gt;")
    .replace(/"/g, "&quot;")
    .replace(/'/g, "&#39;");
}
```

The booking object is created in memory with a generated ODS identifier and the initial status, and the minimum date is set from the browser-local date.

```
currentBooking = {
  id: "ODS" + Math.floor(10000 + Math.random() * 90000),
  name: nameInput.value.trim(),
  service: selectedService,
  date: dateInput.value,
  time: timeInput.value,
  address: addressInput.value.trim(),
  issue: issueInput.value.trim(),
  status: "Booking Requested"
};

const offset = new Date().getTimezoneOffset() * 60000;
dateInput.min = new Date(Date.now() - offset).toISOString().split("T")[0];
```

4.4 Screenshots

Screenshots below are captured from the running index.html using dummy details only; real phone numbers or home addresses are not used.

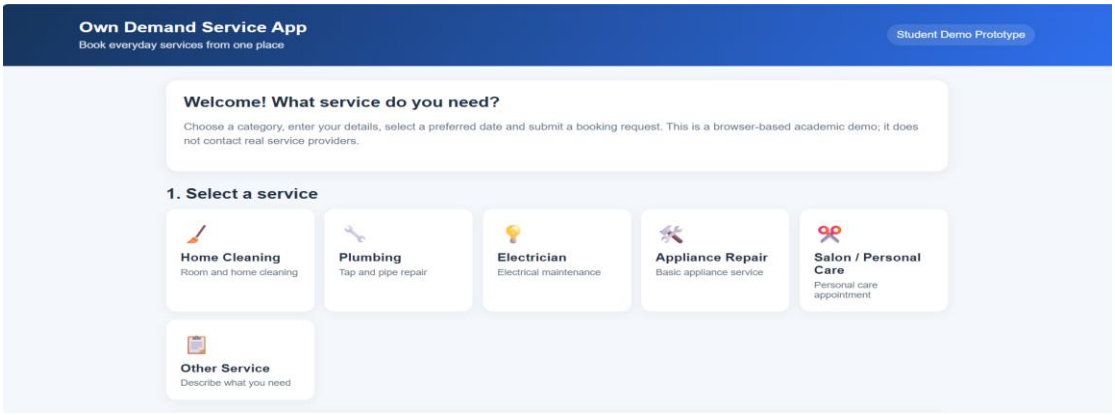


Fig. 4.3: Landing page with service cards

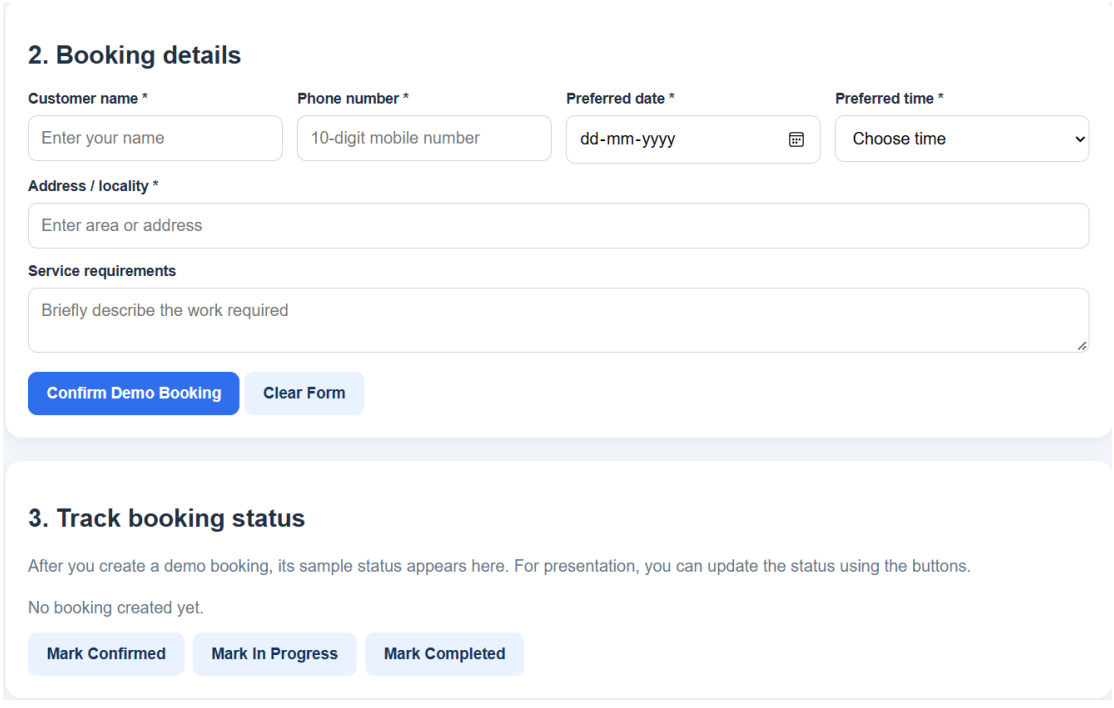


Fig. 4.4: Booking form filled with dummy data

4.5 Error Handling and Input Safety

The interface handles two important submission conditions explicitly. If the user has not selected a category, the script displays a warning and stops. If the phone number is not ten numeric digits, the script also stops before creating the booking. Required fields are additionally supported by HTML validation, and the date control sets a minimum date based on the browser's local date. These checks improve the demonstration flow but are not a substitute for server-side validation.

The confirmation rendering uses an escaping helper for user-entered strings, because untrusted text should not be interpreted as HTML [5]. A future system should avoid building HTML strings where possible and use safe DOM text APIs, enforce server-side length and format constraints, protect against cross-site scripting and injection, and avoid exposing personal data in logs. Validation messages should explain what the user needs to correct without exposing internal implementation details.

4.6 Deployment and Maintenance

The current deployment process is deliberately simple: extract the ZIP archive and open index.html in a modern browser. This makes the project easy to demonstrate without installing dependencies. It also means that there is no hosted URL, central service or persistent store. For a class demonstration, the source archive should remain unchanged and should be tested on the machine that will be used during evaluation.

A future deployment would separate the front-end client from a backend API and database. The website could be hosted through a static hosting service, while the API and database would require environment configuration, access control, monitoring and backup. Releases should be versioned, changes reviewed and test cases rerun after modifications. Secrets must never be placed in client-side JavaScript.

CHAPTER 5: TESTING

5.1 Testing Strategy

Testing is intended to verify that the browser interface responds correctly to common valid and invalid inputs. Manual functional testing is appropriate for this small prototype. Because the project has no automated test suite, backend, database or deployed environment, this report does not claim load testing, penetration testing or end-to-end testing with real providers.

- Functional tests: service selection, form submission, status buttons and reset.
- Validation tests: required fields, phone format and date restriction.
- Negative tests: submitting without a category, status buttons before a booking exists.
- Safety test: special characters in the description are displayed as text.
- Responsive test: layout at narrow widths.

5.2 Test Environment and Preconditions

The test environment should be recorded as part of the final academic evidence: the computer or device type, operating system, browser name and version, screen width used for responsive checks and the date of execution. Tests use fictional names, sample phone numbers and sample addresses.

- Source: the unchanged submitted ZIP, extracted to a local folder.
- Browser: a current version of Chrome, Edge or Firefox.
- Data: fictional details only, for example "Demo User" and "Sample Locality".

5.3 Test Cases and Test Results

Table 5.1 provides a representative set of functional and validation cases. "Expected result" describes the required behaviour; the final column is completed after executing each test on the submitted ZIP.

Table 5.1: Test cases and expected outcomes

Test ID	Scenario	Expected result	Actual result
TC-01	Open index.html in a modern browser	Landing page and service cards display	_____
TC-02	Click each service card	Only the selected card is highlighted	_____
TC-03	Submit without selecting a category	Warning shown; booking not created	_____
TC-04	Leave a required field blank	Browser blocks submission	_____
TC-05	Enter a 9-digit phone number	Submission is rejected	_____
TC-06	Enter letters in the phone field	Submission is rejected	_____
TC-07	Enter exactly 10 digits and valid data	Demo booking summary appears	_____
TC-08	Choose a past date	Date input disallows it via the min attribute	_____
TC-09	Click Mark Confirmed	Displayed status changes to Confirmed	_____
TC-10	Click Mark In Progress	Displayed status changes to In Progress	_____
TC-11	Click Mark Completed	Displayed status changes to Completed	_____
TC-12	Click Clear Form	Form and booking display return to initial state	_____
TC-13	Enter special characters in the description	Text displays as text, not executable markup	_____
TC-14	Resize browser to a narrow width	Cards and form reflow without major overflow	_____
TC-15	Reload after creating a demo booking	In-memory booking is lost; no persistence exists	_____

5.4 Sample Input / Output

Sample input: customer name "Demo User"; phone "9876543210"; category "Home Cleaning"; date set to a future date; time "9 AM - 12 PM"; address "Sample Locality"; issue "Routine room cleaning".

Expected output: a generated ODSxxxxx identifier, a confirmation summary containing the selected category and entered scheduling details, and the initial status "Booking Requested". The identifier is random and will differ between runs.

Invalid input (category): leave the category unselected and submit. Expected output: a warning asks the user to select a service category and no booking is created.

Invalid input (phone): enter fewer than ten digits. Expected output: browser or JavaScript validation prevents booking creation.

5.5 Boundary and Negative Testing

Boundary testing checks values at the edges of validation rules. For the phone field the expected rule is that a trimmed value containing exactly ten digits can proceed. The date control should be tested with today and a future date, while past dates should be blocked by the minimum attribute in supporting browsers.

Table 5.2: Phone-field boundary cases

Input	Expected behaviour	Observed
Empty	Blocked by the required attribute	_____
9 digits	Rejected	_____
10 digits	Accepted	_____
11 digits	Rejected (also limited by the ten-character maximum)	_____
Letters only	Rejected	_____
Mixed alphanumeric	Rejected	_____
Leading / trailing spaces around 10 digits	Accepted after trimming	_____

Negative testing should also cover clicking status controls before a booking exists and clearing the form after a booking has been created. The status handler contains a guard for a missing booking. Browser-level validation can vary slightly across browsers, so the exact message may differ while the outcome should remain consistent.

5.6 Test Evidence and Acceptance Criteria

A test is considered passed only when the observed result matches the expected result on the submitted copy. A screenshot alone does not establish that a test has passed; it should be paired with a test ID and a short note describing the input and expected behaviour. If a test fails, the failure details should be preserved and the defect fixed before retesting.

- Record the test execution date and environment details.
- Pair each screenshot with a test ID.
- Report failures as well as successes.
- Use fictional data in all evidence.

5.7 Test Execution Record Template

Table 5.3 records enough information for another person to reproduce the tests. It is intentionally blank and is completed after execution.

Table 5.3: Test execution record

Field	Value
Date of execution	
Tester	
Device and operating system	
Browser and version	
Screen width used for responsive check	
Test IDs executed	
Pass / fail summary	
Defects found and fixes	
Screenshot references	

CHAPTER 6: RESULTS AND DISCUSSION

6.1 Output Screenshots

The prototype provides a visible service-selection area, a booking form, a confirmation panel and a status panel. The following figures are captured from the submitted index.html using dummy details.

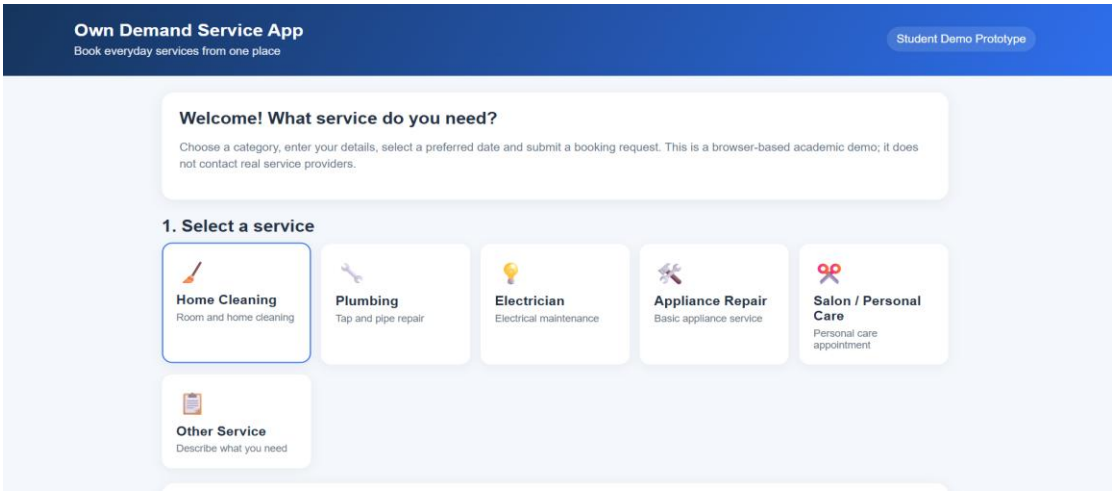


Fig. 6.1: Demo confirmation with generated booking ID

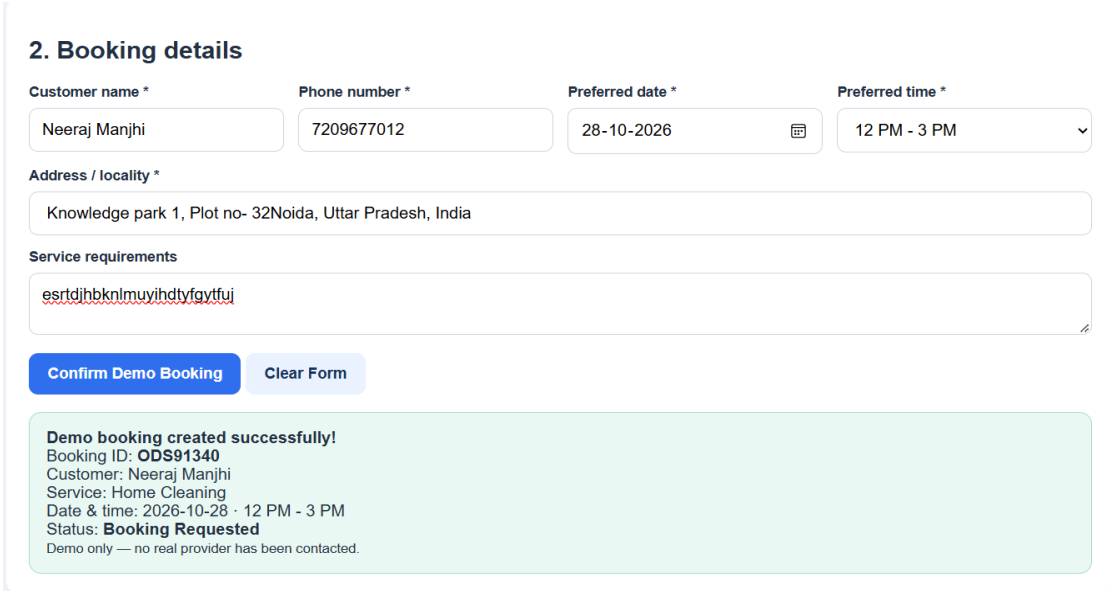


Fig. 6.2: Status panel after a sample status change

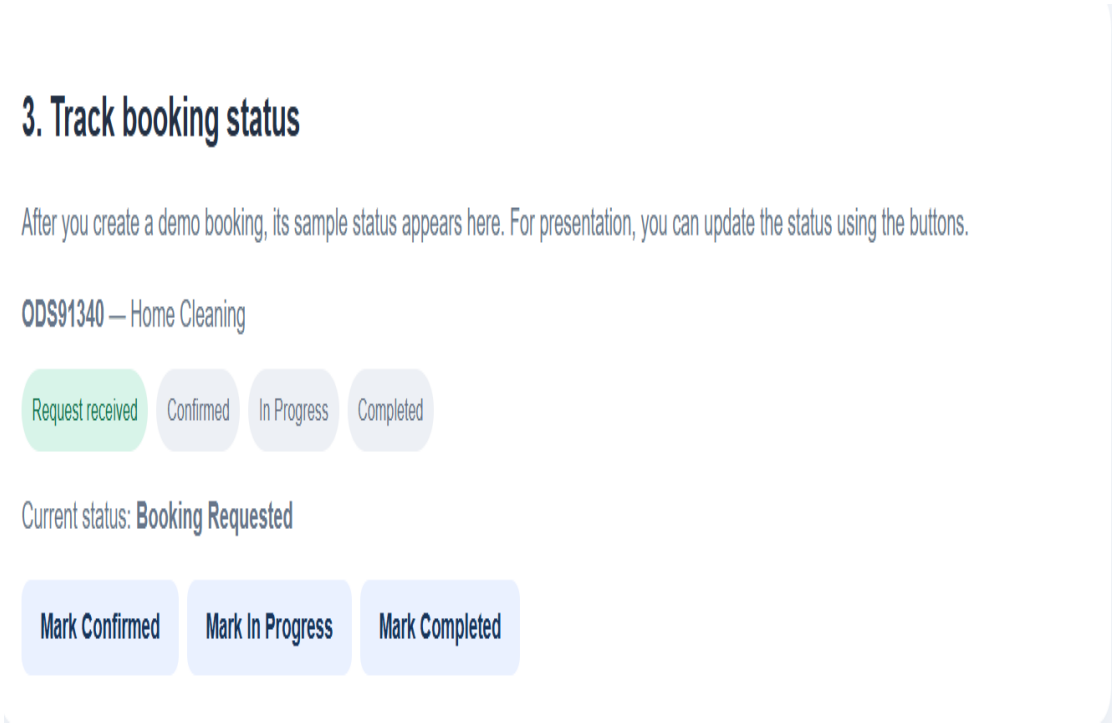


Fig. 6.3: Validation warning for an invalid phone number

6.2 Performance / Accuracy Analysis

The application is a small static page with embedded styling and script. It does not perform network requests, search a provider database or call external APIs. As a result, its main operations are local DOM updates and form validation. No formal timing, throughput, accuracy or user-study measurements have been collected; numerical performance claims would therefore be unsupported.

Accuracy in this project is best interpreted as whether the interface correctly preserves and displays the details entered during a demo submission and whether the validation rules block obvious incomplete inputs. A proper evaluation should execute the test matrix, record browser and device details, repeat cases, and document failures as well as successes.

6.3 Discussion

The prototype demonstrates a coherent first step toward a unified household-service request interface. Its strengths are a straightforward flow, multiple categories, structured fields, a visible identifier, a simulated status path and a responsive layout. These elements can be presented live during project evaluation.

The main limitation is that the status is controlled by demonstration buttons and the booking is stored only in JavaScript memory. There is no real service-provider matching or fulfilment. The interface should therefore be described as a front-end prototype, not a deployed booking platform, because the target problem involves trust, service availability, scheduling and accountability, which require operational processes and a secure backend in addition to a user interface.

6.4 Observed and Unimplemented Capabilities

Table 6.1: Observed and unimplemented capabilities

Capability	Status in prototype	Notes
Service category selection	Implemented (demo)	Six cards with a single active highlight
Required-field and phone validation	Implemented (client-side)	HTML attributes plus a JavaScript check
Past-date prevention	Implemented (client-side)	Minimum date from the browser-local date
Demo booking ID and summary	Implemented (demo)	Random ODS identifier held in memory
Sample status progression	Implemented (demo)	Buttons change the displayed status only
Escaped confirmation text	Implemented	Limited front-end safety measure
Persistent booking storage	Not implemented	Booking is lost on reload
Provider search and matching	Not implemented	No provider directory
Authentication and roles	Not implemented	Anyone can press status buttons
Notifications and payments	Not implemented	No external services

6.5 Risk and Mitigation

Potential future risks include fake provider profiles, unreliable service quality, exposure of customer addresses, inaccurate availability, cancellations and disputes. The current prototype does not process real customer information and should be demonstrated with dummy data.

Table 6.2: Risk and mitigation summary

Area	Current prototype status	Future mitigation / next step
Booking storage	In-memory only	Persist requests in a database through a server API
Provider matching	Not implemented	Search verified providers by category, locality and availability
Authentication	Not implemented	Secure customer/provider accounts and role-based access
Notifications	Not implemented	Send opt-in booking updates by approved channels
Payments	Not implemented	Use a compliant payment provider if payments are in scope
Trust and safety	No provider verification	Verification workflow, reviews, complaint handling and audit trail
Privacy	Demo-only; no secure backend	Minimise collection, protect access and define a retention policy

6.6 Evaluation Metrics for a Future Study

A future evaluation can measure task completion rate, time to submit a request, number of validation errors, number of steps required, perceived ease of use and user confidence in understanding the booking status. Participants could be asked to complete representative tasks such as finding the cleaning category and submitting a plumbing request. Feedback should be gathered with consent and should not require real addresses or phone numbers.

Technical evaluation should measure page load time on target devices, browser compatibility, API response time after a backend is added, booking persistence, concurrent requests and error recovery. Security testing should include access-control checks, input validation, session management and safe handling of personal information. No user study or performance benchmark is claimed for the present prototype.

6.7 Privacy, Trust and Service Quality

A real service platform would process potentially sensitive information, including a person's phone number, location and description of a household problem. Such data should be limited to what is necessary, transmitted securely and visible only to authorised parties. The platform should explain why information is collected, how long it is retained and how a user can request correction or deletion. The student demo should use fictitious details because it has no secure backend or privacy-management controls.

Trust is also a service-design concern. Provider identity checks, clear service descriptions, transparent pricing, reviews linked to completed bookings, complaint resolution and an emergency escalation policy can help users make informed decisions. Any ranking or recommendation mechanism should be explainable. These controls are future requirements rather than features of the submitted ZIP.

CHAPTER 7: CONCLUSION AND FUTURE SCOPE

7.1 Conclusion

The Own Demand Service App mini project addresses a practical scenario: a person unfamiliar with a locality may need a convenient starting point for requesting household services. The submitted front-end prototype groups several service categories in one interface and demonstrates how a customer can enter contact details, choose a preferred schedule, describe a task and view a confirmation summary with a demo booking ID.

The project applies fundamental web development concepts including HTML form controls, CSS layout and responsive styling, JavaScript event handling, input validation and dynamic DOM updates. The work provides a clear basis for discussion of requirements, interface design, testing and the boundary between a prototype and a production service. The implemented result is intentionally limited: it does not contact service providers or save bookings permanently.

7.2 Limitations

- No backend or persistent storage; a page reload clears the booking.
- No authentication, user accounts or roles.
- No provider profiles, availability, matching or dispatch.
- No live status updates; status is changed by demonstration buttons.
- No map integration, reviews, payments or notifications.
- The generated booking ID is a random demo identifier and is not guaranteed to be globally unique.
- Client-side validation can be bypassed and must not be the only control in a real system.
- No formal accessibility audit, usability study or performance evaluation has been performed.

7.3 Future Scope

A practical next version can be developed in stages, each with acceptance criteria. For example, persistent booking storage is complete only when a booking remains available after a page reload and can be retrieved by an authorised account.

Table 7.1: Development roadmap

Phase	Focus	Acceptance criterion
Phase 1	Preserve the prototype as a user-interface proof of concept; gather usability feedback with dummy data	Feedback recorded and design changes listed
Phase 2	Backend API, database schema, server-side validation, customer authentication	Booking persists after reload and is retrievable by an authorised account
Phase 3	Provider registration, identity verification, service areas, schedules, request acceptance	Eligible providers identified by category and locality; unavailable providers represented accurately
Phase 4	Booking history, notifications, cancellation handling, customer support	Delivery outcomes and user preferences handled
Phase 5	Security, accessibility, performance and reliability testing	Test results reviewed before limited pilot use

- Backend API and persistent database.
- Provider registration and verification.
- Locality-based search and availability calendar.
- Customer and provider dashboards.
- Ratings, reviews, complaint resolution and admin oversight.

7.4 Final Remarks

The submitted application should be demonstrated honestly as a browser-based academic prototype. A successful demonstration shows the complete local workflow and explains which parts would need to be built before real customers could rely on the service. This framing presents the project's value without overstating its current capabilities.

REFERENCES

[1] WHATWG, "HTML Living Standard." [Online]. Available: https://html.spec.whatwg.org/. Accessed: 23-Oct-2026.

[2] Mozilla Developer Network, "HTML forms." [Online]. Available: https://developer.mozilla.org/en-US/docs/Learn_web_development/Extensions/Forms. Accessed: 23-Oct-2026.

[3] Mozilla Developer Network, "CSS: Cascading Style Sheets." [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/CSS. Accessed: 23-Oct-2026.

[4] Mozilla Developer Network, "JavaScript Guide." [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide. Accessed: 23-Oct-2026.

[5] OWASP Foundation, "Cross Site Scripting Prevention Cheat Sheet." [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html. Accessed: 23-Oct-2026.

[6] W3C, "Web Content Accessibility Guidelines (WCAG) 2.2." [Online]. Available: https://www.w3.org/TR/WCAG22/. Accessed: 23-Oct-2026.

[7] Own Demand Service App, "Own_Demand_Service_App_Demo: index.html and README.txt," student project source code, 2026.

APPENDIX

Appendix A: Source Code

The submitted source archive contains two files: index.html and README.txt. The HTML file includes page markup, CSS styling and JavaScript logic. The README describes how to run the demo locally. Table A.1 lists the main source elements.

Table A.1: Source elements

Source element	Purpose
HTML header and hero	Project name, introductory note and demo limitation
Service cards	Six selectable service categories
bookingForm	Customer details, preferred date/time, address and task description
selectedService	Stores the currently selected category in memory
currentBooking	Stores the current demo booking object in memory
renderBooking()	Displays confirmation and current status steps
escapeHTML()	Escapes special characters in user-entered text before rendering
data-status buttons	Update the sample status for demonstration
resetBtn handler	Clears form values, selected service and visible booking state

Complete listing of index.html:

neerajmanjhi98/own-demand-service-app

Appendix B: Additional Material

How to run

- Extract Own_Demand_Service_App_Demo.zip to a folder.
- Open the extracted folder and double-click index.html (Chrome, Edge or another modern browser).
- Select one service card and enter dummy customer information.
- Choose a future date and a time window, then click Confirm Demo Booking.
- Show the generated booking ID, summary and initial status.
- Use Mark Confirmed, Mark In Progress and Mark Completed to demonstrate the status view.
- Click Clear Form to return to the initial state.

Suggested viva demonstration

Begin by explaining the user problem and project objective. Demonstrate category selection, required fields, phone validation, date selection, booking creation, status updates and reset. Then explain that the project uses HTML, CSS and JavaScript and that state is held only in browser memory. State clearly that there is no backend, database or real provider communication. Conclude with the proposed next steps: provider directory, secure accounts, persistent booking storage, availability management and notifications.